Thought for 9s ›

Here's the full build sheet in one place, with the updated **10 Ω resistor power input** and **no diodes** version.

Edit

DIY Eurorack BPM Clock Module Build Sheet

1. Project Context

This project is a DIY Eurorack BPM clock module built around an **Arduino Nano R4**, an **I2C OLED display**, an **EC11 rotary encoder with push switch**, and an **SN74AHCT125N quad buffer** for protected clock outputs.

The goal is to create a compact master clock module for a Eurorack system. The module generates steady **0 V / +5 V clock pulses** at a user-selected BPM and provides four synchronized clock outputs:

- **1x A** — one pulse every beat
- **1x B** — a second duplicate one-pulse-per-beat output
- **1/2** — one pulse every two beats
- **1/4** — one pulse every four beats

The **rotary encoder** is the main control:

- Turning the encoder changes the BPM.
- A short press starts or stops the clock.
- Holding the encoder button for about two seconds enters an options/menu mode for additional clock and pattern settings.

The **OLED display** shows the current BPM, run/stop state, and menu information.

The module is powered from the Eurorack case's existing **+5 V rail** using a **16-pin Eurorack power header**. A **10 Ω resistor** is used in series with the incoming +5 V rail for simple filtering and current limiting. This version intentionally omits the LM7805 regulator, ferrite bead, polyfuse, and diode protection.

The **SN74AHCT125N** is used between the Arduino and the output jacks. This buffers the four clock signals and helps protect the Nano R4 from direct exposure to Eurorack patch points. Each clock output also uses a **1 kΩ series resistor** before the jack tip and a **100 kΩ pulldown resistor** to ground.

This build is intended as a practical stripboard prototype that can later be turned into a cleaner PCB or panelized Eurorack module.

---

2. Important Safety Note

This version uses:

- no reverse-polarity diode
- no polyfuse
- no resettable fuse
- no ferrite bead
- no voltage regulator

Because of that, the module depends on correct Eurorack ribbon orientation and a reliable +5 V rail from the case power supply.

Before installing the Nano R4 or SN74AHCT125N, always verify the power rails with a multimeter.

Check:

```
module +5 V rail to GND = about +5.0 V
```

Do not connect the module if the voltage is wrong.

Also, avoid powering the Nano R4 from USB and Eurorack power at the same time unless you add proper power-selection or backfeed protection. For programming, unplug the Eurorack ribbon and power the Nano from USB.

---

3. Main Features

The finished module will provide:

```
BPM-controlled master clock
OLED BPM/status display
```

```
EC11 rotary encoder control
short press = start/stop
2-second hold = options/menu mode
four buffered Eurorack clock outputs
0 V / +5 V output pulses
1x A output
1x B output
1/2 output
1/4 output
```

Recommended first firmware features:

```
BPM adjustment
run/stop
pulse width setting
output mute options
```

Possible later firmware features:

```
swing
pattern mode
clock division options
output inversion
pattern length
manual reset
tap tempo
internal/external sync
```

---

4. Parts List

Main Parts

```
1x Arduino Nano R4
1x SN74AHCT125N, DIP-14
1x DIP-14 socket
1x SSD1306 I2C OLED display
1x EC11 rotary encoder with push switch
4x 3.5 mm mono Eurorack jack sockets
1x 16-pin Eurorack power header
1x 16-pin Eurorack ribbon cable
1x stripboard
female headers for Nano R4, recommended
```

```
hookup wire
panel hardware
```

Resistors

```
1x 10 Ω resistor, preferably 1/2 W, 1/4 W acceptable for tes
4x 1 kΩ resistors for clock output series protection
4x 100 kΩ resistors for output pulldowns
1x 10 kΩ resistor for SN74AHCT125N enable pullup
```

Capacitors

```
1x 100 µF electrolytic capacitor
1x 10 µF capacitor
3–5x 100 nF ceramic capacitors
```

Recommended 100 nF capacitor locations:

```
1x across main +5 V and GND rails
1x directly across SN74AHCT125N pin 14 and pin 7
1x near the OLED power pins, optional
1x near Nano R4 power pins, optional
```

---

5. Board Choice

Use the **Arduino Nano R4**.

It is a good choice for this module because:

```
Nano form factor
5 V logic on normal I/O pins
more memory than classic Nano
fast enough for accurate clock timing
USB-C
suitable for OLED/menu firmware
```

The classic Nano 3.0 would also work, but the Nano R4 has much more memory and processing headroom.

---

6. Output Behavior

Assuming “1x” means one pulse per beat:

Output	Behavior
1x A	pulse every beat
1x B	duplicate pulse every beat
1/2	pulse every 2 beats
1/4	pulse every 4 beats

Recommended pulse level:

```
LOW  = 0 V
HIGH = +5 V
```

Recommended default pulse width:

```
10 ms
```

Good adjustable range:

```
1 ms to 50 ms
```

---

7. Overall Module Block Diagram

```
16-pin Eurorack power header
        |
        | +5 V and GND only
        v
10 Ω input resistor
        |
        v
Stripboard +5 V / GND rails
        |
        +--> Arduino Nano R4
        |
        +--> OLED display
        |
        +--> SN74AHCT125N output buffer
        |
        +--> Encoder pullups / button
```

```
Nano R4
  |
  +--> OLED over I2C
  |
  +--> EC11 encoder inputs
  |
  +--> SN74AHCT125N inputs
            |
            +--> 1 kΩ resistors
                     |
                     +--> Eurorack output jacks
```

---

8. Eurorack 16-Pin Header

Use a **16-pin Eurorack power header** because this build uses the case's existing **+5 V rail**.

With the **red stripe side = -12 V**, the typical 16-pin layout is:

```
Top of header

GATE   GATE     ignore
CV     CV       ignore
+5V    +5V      use
+12V   +12V     ignore
GND    GND      use
GND    GND      use
GND    GND      use
-12V   -12V     red stripe side, ignore

Bottom / red stripe side
```

For this module, connect only:

```
+5 V row -> 10 Ω resistor -> module +5 V rail
GND rows -> module GND rail
```

Do not connect:

```
+12 V
-12 V
CV bus
Gate bus
```

9. Power Layout

This version uses the Eurorack case's +5 V rail directly.

```
16-pin Eurorack +5 V
   |
  10 Ω resistor
   |
module +5 V rail
   |
   +---- Nano R4 5V
   +---- OLED VCC
   +---- SN74AHCT125N pin 14
   |
  100 µF cap to GND
  10 µF cap to GND
  100 nF cap to GND

16-pin Eurorack GND
   |
module GND rail
   |
   +---- Nano R4 GND
   +---- OLED GND
   +---- SN74AHCT125N pin 7
   +---- all jack sleeves
```

Use:

```
10 Ω, 1/4 W minimum
10 Ω, 1/2 W preferred
```

The 10 Ω resistor provides a small amount of filtering and current limiting, but it also causes voltage drop.

Approximate voltage drop:

```
50 mA draw  -> 0.5 V drop
100 mA draw -> 1.0 V drop
```

After the module is running, measure the module's +5 V rail. Ideally it should be close to:

```
4.75 V to 5.0 V
```

If the rail is too low, replace the 10 Ω resistor with a smaller resistor such as:

```
4.7 Ω
```

---

10. Power Capacitors

Place these after the 10 Ω resistor, on the module side:

```
+5 V rail ---- 100 µF ---- GND
+5 V rail ---- 10 µF ----- GND
+5 V rail ---- 100 nF ---- GND
```

Electrolytic capacitor polarity:

```
positive leg -> +5 V rail
negative leg -> GND rail
```

Place one 100 nF ceramic capacitor directly across the SN74AHCT125N power pins:

```
SN74AHCT125N pin 14 -> 100 nF -> SN74AHCT125N pin 7
```

---

11. Arduino Nano R4 Pin Assignment

Use this pin map:

```
Nano R4 5V  -> module +5 V rail
Nano R4 GND -> module GND rail

Nano D2  -> encoder A
Nano D3  -> encoder B
Nano D4  -> encoder switch

Nano A4  -> OLED SDA
Nano A5  -> OLED SCL

Nano D8  -> 1x A clock logic
Nano D9  -> 1x B clock logic
Nano D10 -> 1/2 clock logic
Nano D11 -> 1/4 clock logic

Nano D12 -> SN74AHCT125N buffer enable node
```

12. OLED Wiring

For a typical 4-pin I2C SSD1306 OLED:

```
OLED VCC -> module +5 V rail
OLED GND -> module GND rail
OLED SDA -> Nano R4 A4
OLED SCL -> Nano R4 A5
```

Use the Nano R4's normal A4/A5 I2C pins for a normal 5 V OLED module.

Do not use the Nano R4 Qwiic connector unless you are specifically using a 3.3 V Qwiic-compatible display.

---

13. EC11 Rotary Encoder Wiring

Typical EC11 pins:

```
A
C / common
B

SW1
SW2
```

Wire it like this:

```
Encoder A      -> Nano D2
Encoder B      -> Nano D3
Encoder common -> GND

Switch pin 1   -> Nano D4
Switch pin 2   -> GND
```

Use Arduino internal pullups in firmware:

```
pinMode(2, INPUT_PULLUP);
pinMode(3, INPUT_PULLUP);
pinMode(4, INPUT_PULLUP);
```

Optional hardware debounce capacitors:

```
D2 to GND -> 10 nF
D3 to GND -> 10 nF
D4 to GND -> 100 nF
```

---

14. SN74AHCT125N Pinout

Place the SN74AHCT125N with the notch facing up.

```
            SN74AHCT125N
            notch up


        ┌──────────────┐
 /OE1  1|              |14 VCC
  A1   2|              |13 /OE4
  Y1   3|              |12 A4
 /OE2  4|              |11 Y4
  A2   5|              |10 /OE3
  Y2   6|              |9  A3
 GND   7|              |8  Y3
        └──────────────┘
```

Power pins:

```
Pin 14 VCC -> +5 V rail
Pin 7 GND  -> GND rail
100 nF capacitor directly between pin 14 and pin 7
```

---

15. SN74AHCT125N Buffer Enable Wiring

Recommended safer boot behavior:

```
SN74AHCT125N pins 1, 4, 10, 13 -> common enable node
common enable node -> 10 kΩ resistor -> +5 V rail
common enable node -> Nano D12
```

The SN74AHCT125N output-enable pins are active low.

Behavior:

```
D12 HIGH = outputs disabled
D12 LOW  = outputs enabled
```

This keeps the outputs disabled during boot until the firmware intentionally enables them.

Simpler version, not recommended:

```
SN74AHCT125N pins 1, 4, 10, 13 -> GND
```

That works, but the outputs may briefly glitch during boot.

---

16. Clock Output Buffer Wiring

Output 1: 1x A

```
Nano D8 -> SN74AHCT125N pin 2
SN74AHCT125N pin 3 -> 1 kΩ resistor -> jack 1 tip
jack 1 sleeve -> GND
jack 1 tip -> 100 kΩ resistor -> GND
```

Output 2: 1x B

```
Nano D9 -> SN74AHCT125N pin 5
SN74AHCT125N pin 6 -> 1 kΩ resistor -> jack 2 tip
jack 2 sleeve -> GND
jack 2 tip -> 100 kΩ resistor -> GND
```

Output 3: 1/2 Clock

```
Nano D10 -> SN74AHCT125N pin 9
SN74AHCT125N pin 8 -> 1 kΩ resistor -> jack 3 tip
jack 3 sleeve -> GND
jack 3 tip -> 100 kΩ resistor -> GND
```

Output 4: 1/4 Clock

```
Nano D11 -> SN74AHCT125N pin 12
SN74AHCT125N pin 11 -> 1 kΩ resistor -> jack 4 tip
jack 4 sleeve -> GND
jack 4 tip -> 100 kΩ resistor -> GND
```

Each jack output stage looks like this:

```
SN74AHCT125N output
        |
       1 kΩ
        |
```

```
          +------ jack tip
          |
        100 kΩ
          |
         GND

jack sleeve -> GND
```

---

17. Full Connection Table

Power

From	To
16-pin +5 V	10 Ω resistor
10 Ω resistor	module +5 V rail
16-pin GND	module GND rail
+5 V rail	Nano R4 5V
GND rail	Nano R4 GND
+5 V rail	OLED VCC
GND rail	OLED GND
+5 V rail	SN74AHCT125N pin 14
GND rail	SN74AHCT125N pin 7
GND rail	all jack sleeves

Controls and Display

Part	Connection
OLED SDA	Nano A4
OLED SCL	Nano A5

Encoder A	Nano D2
Encoder B	Nano D3
Encoder common	GND
Encoder switch 1	Nano D4
Encoder switch 2	GND

Outputs

Clock	Nano pin	AHCT input	AHCT output	Jack
1x A	D8	pin 2	pin 3	jack 1 tip via 1 kΩ
1x B	D9	pin 5	pin 6	jack 2 tip via 1 kΩ
1/2	D10	pin 9	pin 8	jack 3 tip via 1 kΩ
1/4	D11	pin 12	pin 11	jack 4 tip via 1 kΩ
Enable	D12	pins 1, 4, 10, 13	—	—

---

18. Suggested Stripboard Physical Layout

A practical left-to-right layout:

```
┌──────────────────────────────────────────────────────────────┐
│ +5V rail ═══════════════════════════════════════════════
│ GND rail ═══════════════════════════════════════════════
│
│ [16-pin power] -- [10Ω] -- +5V rail
│       │
│       └──────────── GND rail
```

```
|
|        [ Nano R4 ]
|        D2  -> encoder A
|        D3  -> encoder B
|        D4  -> encoder switch
|        A4  -> OLED SDA
|        A5  -> OLED SCL
|        D8  -> AHCT pin 2
|        D9  -> AHCT pin 5
|        D10 -> AHCT pin 9
|        D11 -> AHCT pin 12
|        D12 -> AHCT enable node
|
|                         [ SN74AHCT125N ]
|                         pin 14 -> +5V
|                         pin 7  -> GND
|                         pins 1,4,10,13 -> D12 + 10k to +5V
|
|                         pin 3  -> 1k -> jack 1 tip
|                         pin 6  -> 1k -> jack 2 tip
|                         pin 8  -> 1k -> jack 3 tip
|                         pin 11 -> 1k -> jack 4 tip
|
| [OLED header]       [EC11 encoder]       [1xA][1xB][/2][/4
└──────────────────────────────────────────────────────────
```

---

19. Suggested Front Panel Layout

A simple panel arrangement:

```
┌──────────────────────┐
│      OLED SCREEN     │
│                      │
│        BPM           │
│     [ encoder ]      │
│                      │
│  1xA   1xB   /2  /4  │
│  jack  jack jack jack│
└──────────────────────┘
```

Suggested width:

```
6 HP  = possible but tight
8 HP  = comfortable
10 HP = easiest for hand wiring
```

For a first stripboard build, **8 HP or 10 HP** is recommended.

---

20. Firmware Behavior

Main screen:

```
BPM: 120.0
RUN or STOP
1x  1/2  1/4 indicators
```

Encoder behavior:

```
Rotate encoder: change BPM
Short click: start / stop clock
Hold 2 seconds: enter options/menu mode
```

Clock timing:

```
beat_period_us = 60,000,000 / BPM
```

Example:

```
120 BPM = 500,000 microseconds per beat
```

Clock output behavior:

```
Every beat:
  1x A fires
  1x B fires

Every 2 beats:
  1/2 fires

Every 4 beats:
  1/4 fires
```

The firmware should use non-blocking timing with `micros()` rather than `delay()`.

---

21. Starter Firmware Skeleton

This starter code provides:

```
encoder BPM control
short press start/stop
2-second hold menu toggle
four divided clock outputs
SN74AHCT125N enable control on D12
OLED status display
```

```
#include <Arduino.h>
#include <Wire.h>
#include <U8x8lib.h>

// OLED constructor.
// Adjust if your display type is different.
U8X8_SSD1306_128X64_NONAME_HW_I2C display(U8X8_PIN_NONE);

// Encoder pins
const uint8_t ENC_A  = 2;
const uint8_t ENC_B  = 3;
const uint8_t ENC_SW = 4;

// Clock output logic pins
const uint8_t OUT_1X_A    = 8;
const uint8_t OUT_1X_B    = 9;
const uint8_t OUT_HALF    = 10;
const uint8_t OUT_QUARTER = 11;

// SN74AHCT125N output enable node.
// Active low: LOW = enabled, HIGH = disabled.
const uint8_t BUF_EN = 12;

const uint8_t outputPins[] = {
  OUT_1X_A,
  OUT_1X_B,
  OUT_HALF,
  OUT_QUARTER
};

float bpm = 120.0;
bool running = false;
bool inMenu = false;
```

```
uint32_t beatCount = 0;
uint32_t nextBeatUs = 0;
uint32_t pulseWidthUs = 10000; // 10 ms

bool outputHigh[4] = {false, false, false, false};
uint32_t outputOffTime[4] = {0, 0, 0, 0};

int lastEncState = 0;

bool lastButtonReading = HIGH;
bool buttonState = HIGH;
uint32_t lastDebounceMs = 0;
uint32_t buttonDownMs = 0;
bool longPressHandled = false;

uint32_t lastDisplayMs = 0;

uint32_t beatPeriodUs() {
  return (uint32_t)(60000000.0 / bpm);
}

void fireOutput(uint8_t index) {
  digitalWrite(outputPins[index], HIGH);
  outputHigh[index] = true;
  outputOffTime[index] = micros() + pulseWidthUs;
}

void fireBeat() {
  fireOutput(0); // 1x A
  fireOutput(1); // 1x B

  if (beatCount % 2 == 0) {
    fireOutput(2); // 1/2
  }

  if (beatCount % 4 == 0) {
    fireOutput(3); // 1/4
  }

  beatCount++;
}
```

```
void allOutputsLow() {
  for (uint8_t i = 0; i < 4; i++) {
    digitalWrite(outputPins[i], LOW);
    outputHigh[i] = false;
  }
}

void handleClock() {
  uint32_t now = micros();

  if (running) {
    while ((int32_t)(now - nextBeatUs) >= 0) {
      fireBeat();
      nextBeatUs += beatPeriodUs();
    }
  }

  for (uint8_t i = 0; i < 4; i++) {
    if (outputHigh[i] && (int32_t)(now - outputOffTime[i]) >
      digitalWrite(outputPins[i], LOW);
      outputHigh[i] = false;
    }
  }
}

void handleEncoder() {
  int a = digitalRead(ENC_A);
  int b = digitalRead(ENC_B);
  int encState = (a << 1) | b;

  if (encState != lastEncState) {
    if ((lastEncState == 0b00 && encState == 0b01) ||
        (lastEncState == 0b01 && encState == 0b11) ||
        (lastEncState == 0b11 && encState == 0b10) ||
        (lastEncState == 0b10 && encState == 0b00)) {
      if (!inMenu) bpm += 1.0;
    } else {
      if (!inMenu) bpm -= 1.0;
    }

    if (bpm < 20.0) bpm = 20.0;
    if (bpm > 300.0) bpm = 300.0;
```

```
    lastEncState = encState;
  }
}

void handleButton() {
  bool reading = digitalRead(ENC_SW);
  uint32_t nowMs = millis();

  if (reading != lastButtonReading) {
    lastDebounceMs = nowMs;
  }

  if ((nowMs - lastDebounceMs) > 30) {
    if (reading != buttonState) {
      buttonState = reading;

      if (buttonState == LOW) {
        buttonDownMs = nowMs;
        longPressHandled = false;
      } else {
        uint32_t pressLength = nowMs - buttonDownMs;

        if (!longPressHandled && pressLength < 2000) {
          running = !running;

          if (running) {
            beatCount = 0;
            nextBeatUs = micros();
          } else {
            allOutputsLow();
          }
        }
      }
    }

    if (buttonState == LOW && !longPressHandled) {
      if ((nowMs - buttonDownMs) >= 2000) {
        inMenu = !inMenu;
        longPressHandled = true;
      }
    }
  }
```

```
  lastButtonReading = reading;
}

void updateDisplay() {
  uint32_t nowMs = millis();
  if (nowMs - lastDisplayMs < 100) return;
  lastDisplayMs = nowMs;

  display.clear();

  display.setCursor(0, 0);
  display.print(inMenu ? "OPTIONS" : "BPM CLOCK");

  display.setCursor(0, 2);
  display.print("BPM: ");
  display.print(bpm, 1);

  display.setCursor(0, 4);
  display.print(running ? "RUN " : "STOP");

  display.setCursor(0, 6);
  display.print("PW:");
  display.print(pulseWidthUs / 1000);
  display.print("ms");
}

void setup() {
  pinMode(BUF_EN, OUTPUT);
  digitalWrite(BUF_EN, HIGH); // Disable AHCT outputs during

  pinMode(ENC_A, INPUT_PULLUP);
  pinMode(ENC_B, INPUT_PULLUP);
  pinMode(ENC_SW, INPUT_PULLUP);

  for (uint8_t i = 0; i < 4; i++) {
    pinMode(outputPins[i], OUTPUT);
    digitalWrite(outputPins[i], LOW);
  }

  lastEncState = (digitalRead(ENC_A) << 1) | digitalRead(ENC

  display.begin();
  display.setFont(u8x8_font_chroma48medium8_r);
```

```
  allOutputsLow();

  digitalWrite(BUF_EN, LOW); // Enable AHCT outputs after se

  updateDisplay();
}

void loop() {
  handleEncoder();
  handleButton();
  handleClock();
  updateDisplay();
}
```

---

22. Pre-Power Checklist

Before installing the Nano R4 or SN74AHCT125N:

```
1. Check continuity:
   +5 V rail is not shorted to GND.

2. Confirm the 16-pin power header orientation:
   red stripe side = -12 V.

3. Confirm that only +5 V and GND are connected.

4. Plug the module into Eurorack power with no ICs installed

5. Measure:
   module +5 V rail to GND = about +5.0 V.

6. Confirm:
   +12 V is not connected to the +5 V rail.
   -12 V is not connected to the GND rail.
   CV bus is not connected.
   Gate bus is not connected.

7. Power down.

8. Install the SN74AHCT125N.
```

```
9. Install the Nano R4.

10. Power from USB first for firmware testing, with Eurorack

11. Then test from Eurorack +5 V.

12. Measure the running +5 V rail after the 10 Ω resistor.
    Ideally it should be between about 4.75 V and 5.0 V.
```

---

23. First Prototype Testing Order

Recommended testing sequence:

```
1. Build the Nano R4 + OLED + encoder section only.
2. Upload firmware over USB.
3. Confirm the display works.
4. Confirm encoder rotation changes BPM.
5. Confirm short click starts/stops.
6. Confirm 2-second hold enters menu/options mode.
7. Add the SN74AHCT125N.
8. Test the four output pins with LEDs or a multimeter.
9. Add the 1 kΩ output resistors and jacks.
10. Test with another Eurorack module.
11. Verify clock division:
    1x A = every beat
    1x B = every beat
    1/2 = every 2 beats
    1/4 = every 4 beats
```

---

24. Final Build Summary

The final version of this build uses:

```
Arduino Nano R4
16-pin Eurorack header
case-provided +5 V rail
10 Ω resistor in series with +5 V input
no LM7805
no ferrite bead
no polyfuse
no diodes
```

```
OLED on A4/A5
EC11 encoder on D2/D3/D4
SN74AHCT125N powered from +5 V
SN74AHCT125N enable node controlled by D12
D8/D9/D10/D11 feeding the four AHCT inputs
AHCT outputs through 1 kΩ resistors to jacks
100 kΩ pulldown from each jack tip to ground
jack sleeves connected to ground
```

The module outputs four clean 0 V / +5 V clock signals:

```
1x A
1x B
1/2
1/4
```

The module is controlled by one encoder:

```
turn = change BPM
short press = start/stop
hold 2 seconds = options/menu
```

This is a solid first stripboard version of a Eurorack BPM clock module and a good foundation for a future PCB/panel version.